# 16 Implementing an OAuth 2 client

This chapter covers

- Implementing an OAuth 2 login
- Implementing a Spring Security OAuth 2 client
- Using the client credentials grant type

Often, it is necessary to implement communication between backend applications, especially for backend apps involving multiple services. In such cases, when systems have authentication and authorization built over OAuth 2, it's recommended that you authenticate calls between apps using the same approach. While developers use HTTP Basic and API Key authentication methods (chapter 6) for simplicity in some cases to keep the system consistent and more secure, using the OAuth 2 client credentials grant type is the preferred option.

Remember the OAuth 2 actors (figure 16.1)? We discussed the authorization server in chapter 14 and the resource server in chapter 15. This chapter is dedicated to the client. We'll discuss how to use Spring Security to implement an OAuth 2 client and when and how a backend app becomes a client in an OAuth 2 system.

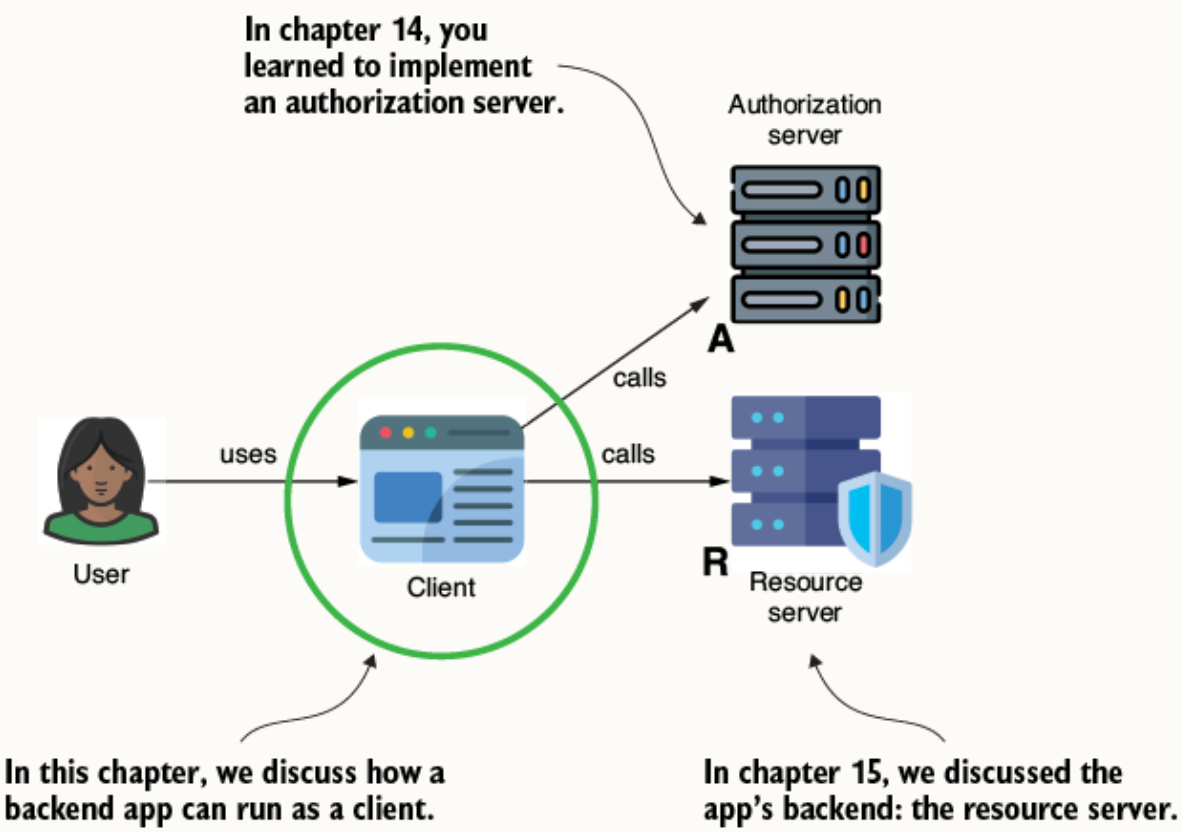



Figure 16.1 The OAuth2 actors. In this chapter, we discuss the client and how a backend app can act as a client in a system with authentication and authorization designed as OAuth 2.

Okay, maybe figure 16.1 doesn't fully illustrate what we will talk about. We'll begin by discussing login for the user, but we'll also focus on how to make a backend app a client for another backend app. Backend apps designed with Spring Security may become clients as well. Figure 16.2 shows the other case that we'll discuss here. In the current chapter, we'll solve the problem of implementing the communication between two backend apps, making one of them an actual OAuth 2 client. In such a case, we need to use Spring Security to build an OAuth 2 client.

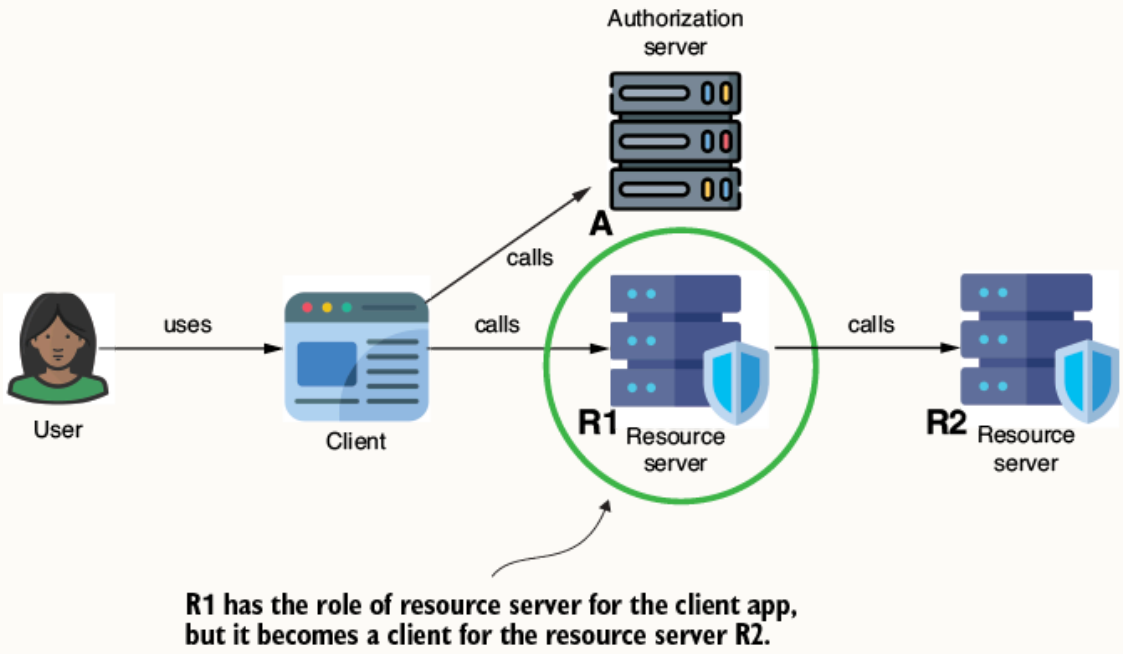



Figure 16.2 A backend app may become a client for another backend app. We discuss this case in the current chapter.

Section 16.1 discusses how to easily implement an OAuth 2 login for a Spring MVC web app using Spring Security. We'll use an external authorization server provider, such as Google and GitHub. You'll learn how to implement a login for your app where users can authenticate using their Google or GitHub credentials. Using the same approach, you can implement such a login with a custom (self-owned) authorization server.

In section 16.2, we employ a client's custom implementation through a service and discuss using the client credentials grant type.

## 16.1 Implementing OAuth 2 login

This section discusses how to implement an OAuth 2 login for your Spring web app. With Spring Boot, it's a piece of cake to configure the authentication for standard cases (cases where the authorization server fulfills the OAuth 2 and OpenID Connect specifications correctly). We'll begin with a classic case (which you can use with most well-known providers such as Google, GitHub, Facebook, and Okta).

Then I'll show you what's behind the scenes of the self-provided configuration so you can also cover custom cases. At the end of this section, you'll be able to implement login for your Spring web app with any OAuth 2 provider and even allow your users to choose between various providers when authenticating.

### 16.1.1 Implementing authentication with a common provider

In this section, we'll implement the simplest login case, allowing our app's users to log in using only one provider. For this demonstration, I chose Google as our user's authentication provider.

We begin by adding a few resources to our project to implement a simple Spring web app with the mentioned login capabilities. Listing 16.1 shows the dependencies for the demo app. You can find the app in this example in project ssia-ch16-ex1. You'll recognize a new dependency we haven't used in previous chapters: *the OAuth 2 client dependency*.

Listing 16.1 Dependencies needed for our demonstration

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-client
  ➥</artifactId>                              ①
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

① The only new dependency you observe is the OAuth 2 client dependency. We need this dependency for all the OAuth 2 client capabilities we configure in the project.

If you need a refresher on building web apps with Spring Boot, chapters 7 and 8 of *Spring Start Here* (Manning, 2020), another book I wrote, should help you remember these skills quickly. The following code snippet shows the simple controller of our demo web app, which only has a home page:

```
@Controller
public class HomeController {

 @GetMapping("/")
 public String home() {
  return "index.html";
```

```
  }
}
```

The next code snippet shows the small demo HTML page we expect to access once the authentication ends successfully:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
</head>
<body>
    <h1>Home</h1>
</body>
</html>
```

Listing 16.2 presents the configuration for OAuth 2 login as an authentication method for the web app. Configuring the app in such a way will automatically follow the authorization code grant type redirecting the user to log in on a specific authorization server and redirecting it back once the authentication succeeds. This process follows precisely what we discussed in chapters 13 through 15 and demonstrated multiple times in these chapters using cURL.

Listing 16.2 Configuring the OAuth 2 login

```
@Configuration
public class SecurityConfig {

  @Bean
  public SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {

    http.oauth2Login(Customizer.withDefaults());        ①

    http.authorizeHttpRequests(
      c -> c.anyRequest().authenticated());

    return http.build();
  }
}
```

① To configure authentication as OAuth 2 login, we use the oauth2Login() method.

I bet you are thinking, Shouldn't we still have to fill in all those details we learned in chapters 13 through 15, such as the authorization URL, token URL, client ID, client secret, and so on? Yes, all these details are needed. Fortunately, Spring Security can help you again. If your app uses one of the providers Spring Security considers to be well known, most of these details are prefilled. You only need to configure the app's client credentials. Spring Security considers the following providers as well known:

- Google
- GitHub
- Okta
- Facebook

Spring Security preconfigures the details for these providers in the `CommonOAuth2Provider` class. So if you use any of these, you only need to configure the client credentials in your application properties, which works. The following code snippet shows the two properties you need to configure the client ID and the client secret when using Google (I truncated my credentials' values):

```
spring.security.oauth2.client.registration.google.client-id=790...
spring.security.oauth2.client.registration.google.client-secret=GOC...
```

I implied here that you have registered your app in the Google Developer Console—that's where you get your app's unique set of credentials from. If you haven't done this before, and you want to configure authentication using Google for your app, you can find Google's detailed documentation on how to register your OAuth 2 app with Google at **http://mng.bz/eEvz**. Figure 16.3 shows how the app displays the Google login when properly configuring this well-known provider.

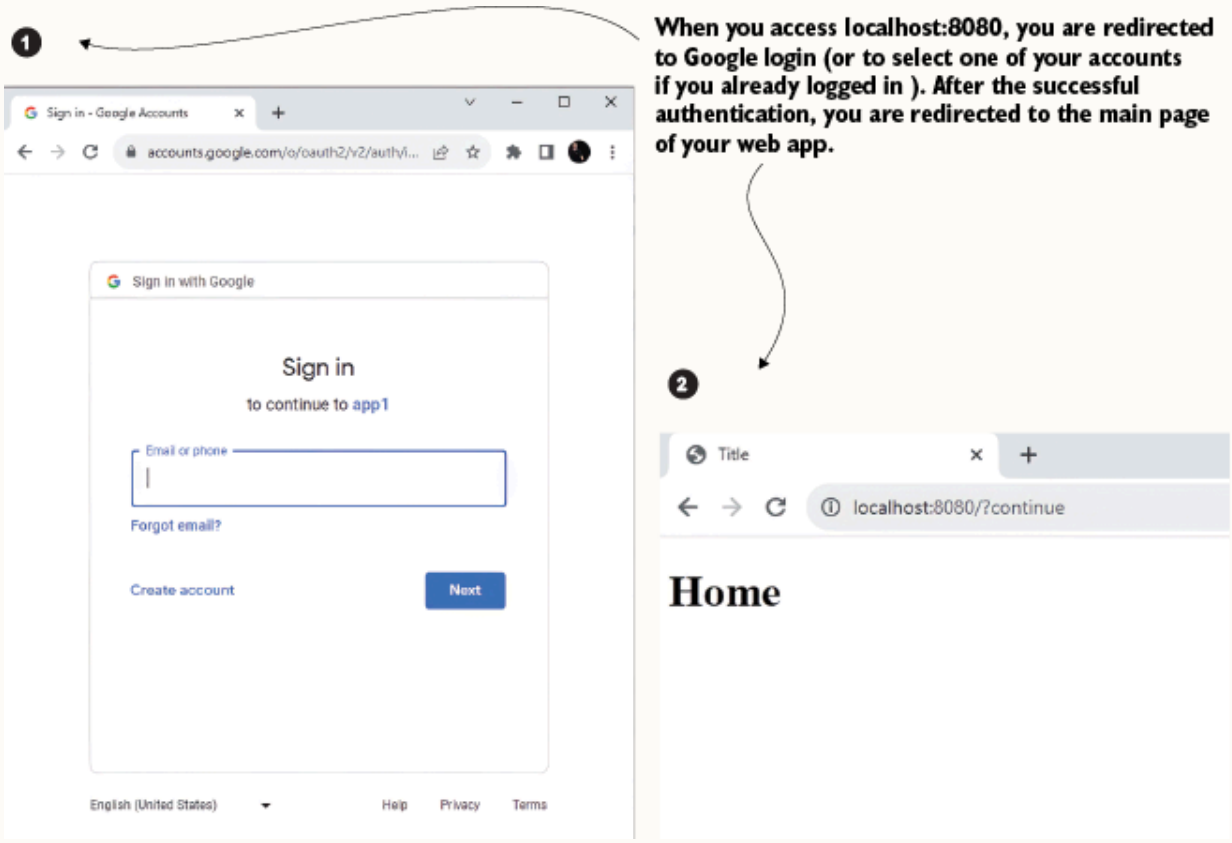



Figure 16.3 When accessing the app in a web browser, the browser redirects you to the Google login page. If you authenticate with Google correctly, you are redirected back to your app's main page.

### 16.1.2 Giving the user more possibilities

I'm sure you've surfed the internet enough by now to observe that many apps offer more than one way for the user to log in. Sometimes you can even choose among four or five providers to log into an app. This approach is advantageous because not all of us already have an account with a social network. Some have a Facebook account, but others prefer to use LinkedIn. Some developers prefer to log in using their GitHub account, but others use their Gmail address.

With Spring Security, you can make this work straightforwardly, even by using multiple providers. Say I want to enable my app's users to log in with either Google or GitHub. I only need to configure the credentials for both providers similarly. The following snippet shows the properties required in the application.properties file to add GitHub as an authentication method. Remember that you must keep those we already configured for Google in section 16.1.1:

```
spring.security.oauth2.client.registration.github.client-id=03...
spring.security.oauth2.client.registration.github.client-secret=c5d...
```

Similar to any other provider, you have to register your app first, configuring the client ID and secret in the application.properties file. The approach to registering the app dif-

fers from one provider to another. For GitHub, you find the documentation that tells you how to register an app at **http://mng.bz/p1YG**.

Before asking you to authenticate, the app gives you two login options: the ones we previously configured (figure 16.4). You must choose either Google or GitHub to log in. After you choose your preferred provider, the app redirects you to that provider's specific authentication page.

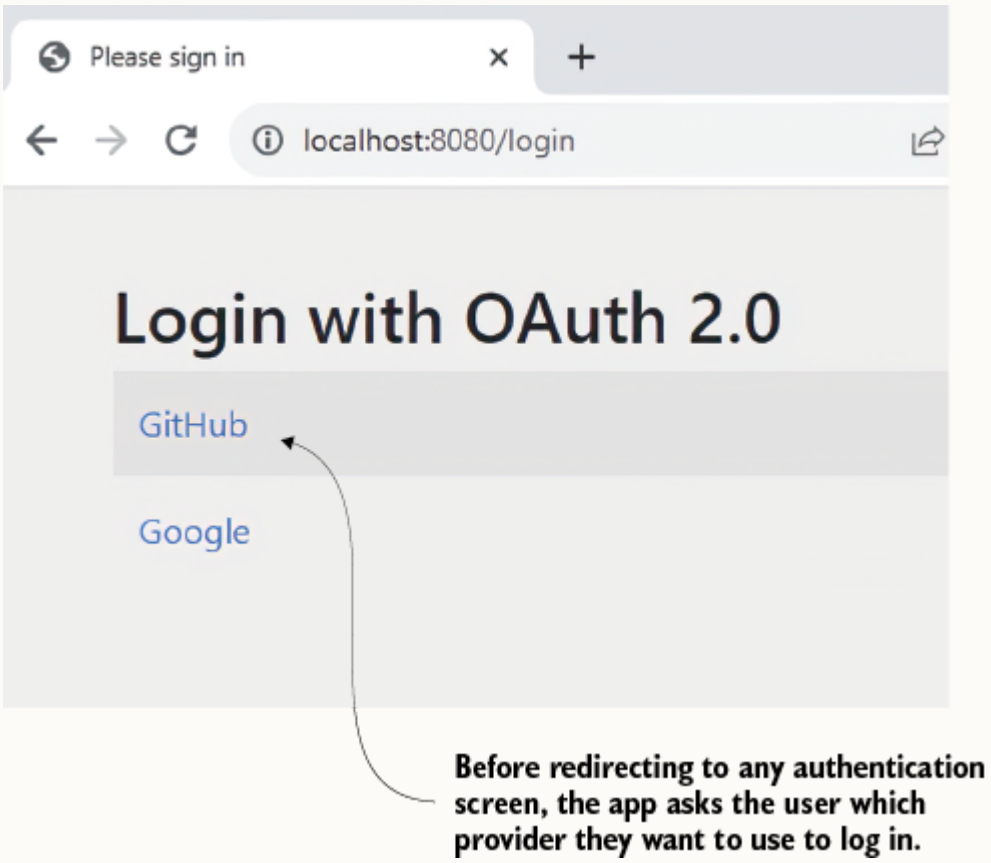



Figure 16.4 The app allows the user to choose between GitHub and Google when authenticating in the app.

### 16.1.3 Using a custom authorization server

Spring Security defines a list of four common providers, as discussed in sections 16.1.1 and 16.1.2. But what if you want to use a provider that is not on the common providers list? You have many other alternatives, such as LinkedIn, Twitter, Yahoo, and others. You might want to use a custom authorization server you built, as you learned in chapter 14.

You can configure an OAuth 2 login with any provider, including a custom one you built. In this section, we're going to use an authorization server we built in chapter 14 to show the configuration of a custom OAuth 2 login. To make your learning easier and also keep the examples separate, I have copied the content of project ssia-ch14-ex1 that we discussed in chapter 14 into a project for this chapter, which I named ssia-ch16-ex1-as.

We only need to ensure that our client configuration matches what we want to implement in this chapter. Listing 16.2 shows the registered client configured in our authorization server. The most important thing here is to make sure that the redirect URI matches the one we expect for our application for which we'll implement the login:

```
http://localhost:8080/login/oauth2/code/my_authorization_server
```

Figure 16.5 analyzes the anatomy of the redirect URI. Observe that the standard redirect URI uses the /login/oauth2/code path followed by the name of the authorization server. In this example, the name I gave to the authorization server is `my_authorization_server`.

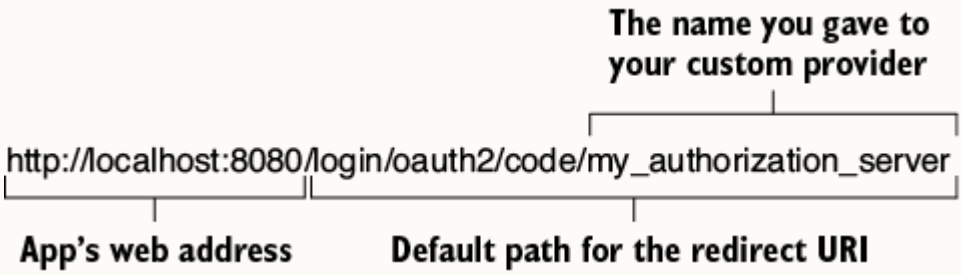



Figure 16.5 The standard redirect URI format. The last part of its path is the provider's name.

The next listing shows the configuration part from the authorization server, which registers the client details. You need these details later in this section; we'll configure them at the app side as well.

Listing 16.3 The client details registered on the authorization server side

```
@Bean
public RegisteredClientRepository registeredClientRepository() {
  var registeredClient = RegisteredClient
    .withId(UUID.randomUUID().toString())
    .clientId("client")
    .clientSecret("secret")
    .clientAuthenticationMethod(
      ClientAuthenticationMethod.CLIENT_SECRET_BASIC)
    .authorizationGrantType(
      AuthorizationGrantType.AUTHORIZATION_CODE)
    .redirectUri(
     "http://localhost:8080/login/oauth2/code/my_authorization_server")
    .scope(OidcScopes.OPENID)
    .build();
```

```
    return new InMemoryRegisteredClientRepository(registeredClient);
}
```

Remember, you can't start two apps using the same port number on the same system. Because the web app uses port 8080, we must change the authorization server's port to another one. As presented in the following code snippet, I chose 7070 for this example and configured it in the application.properties file:

```
server.port=7070
```

We can now move on to the web app's configuration. Because we used a common provider in our examples from sections 16.1.1 and 16.1.2, we didn't have to define it. Spring Security already knows all the details it needs about common providers. However, we need to configure a few things to use a different provider. Spring Security needs to know the following (as discussed in chapters 13 and 14):

- The authorization endpoint for the provider to know where to redirect the user during the authorization code flow
- The token endpoint that the app has to call to get an access token
- The key set endpoint the app needs to validate the access tokens

The good news is that if your provider (authorization server) fulfills the OpenID Connect protocol correctly, you only need to configure the issuer URI. The app then uses the issuer URI to find all details it needs, such as the authorization, token, and the key set URI. If the authorization server doesn't fulfill the OpenID Connect protocol, you'll have to configure these three details in the application.properties file explicitly.

As the authorization servers we built in chapter 14 implement the OpenID Connect protocol correctly, we can rely on the issuer URI. The next code snippet shows how to configure the issuer URI. Observe that I gave a name to the provider. For this example, I chose to identify it with the name `my_authorization_server`, but you can choose any name to identify your provider:

```
spring.security.oauth2.client.provider.my_authorization_server.issuer-uri=http://127.0.0.1:7070
```

**NOTE** We run both apps, the authorization server and the web app we use on the local system. Running these apps on the same system and accessing them from the browser may cause problems with the cookies the browser uses to store the user's session. For

this reason, I recommend you use the IP address `"127.0.0.1"` to refer to one app and the DNS name `"localhost"` to refer to the other. Even if the two are identical from a networking point of view and they refer to the same system (the local system), they will be considered different by the browser, which will in this way be able to manage the sessions correctly. In this example, I use `"127.0.0.1"` to refer to the authorization server and `"localhost"` for the web app.

Listing 16.4 shows the client registration configuration. Apart from declaring who the provider is, the client registration is also a bit longer than the one we wrote in sections 16.1.1 and 16.1.2, where we used common providers. Aside from the client ID and client secret, you also need to fill in the following:

- *The provider name*—A name you give to the provider you want to use in case it's not common.
- *The client authentication meeting*—The app's authentication method to call the provider's secured endpoints (usually HTTP Basic).
- *The redirect URI*—The URI the app expects the provider to redirect the user to after correct authentication. This URI has to match one of those registered on the authorization server side (see listing 16.3).
- *The web app's requested scope*—The scope that the web app requests can only be one of those registered at the authorization server side (see listing 16.3).

**Listing 16.4 The client registration configuration**

```
spring.security.oauth2.client.
➥registration.my_authorization_server
.client-id=client                                          ①
spring.security.oauth2.client.
➥registration.my_authorization_server.
➥client-name=Custom                                       ②
Spring.security.oauth2.client.
➥registration.my_authorization_server.
➥client-secret=secret                                     ③
spring.security.oauth2.client.
➥registration.my_authorization_server.
➥provider=my_authorization_server                         ④
spring.security.oauth2.client.
➥registration.my_authorization_server.
➥client-authentication-method=client_secret_basic         ⑤
spring.security.oauth2.client.
➥registration.my_authorization_server.redirect-uri=
➥http://localhost:8080/login/oauth2/code/my_authorization_server   ⑥
```

```
spring.security.oauth2.client.
➥registration.my_authorization_server.scope[0]=openid                ⑦
```

① The client ID registered at the authorization server side

② The display name of the client

③ The client secret registered at the authorization server side

④ The name of the custom provider

⑤ The app's authentication method to call the provider's protected endpoints

⑥ The URI the provider redirects the user to after successful authentication

⑦ The scope the app requests

You can start the authorization server and the web application. Remember, you must first start the authorization server. When the web app starts, it will call the issue URI to get the rest of the details it needs. Once you start both apps, access the web app in the browser using its address http://localhost:8080. Figure 16.6 shows that the custom provider now appears in the list and can be chosen by the user to authenticate.

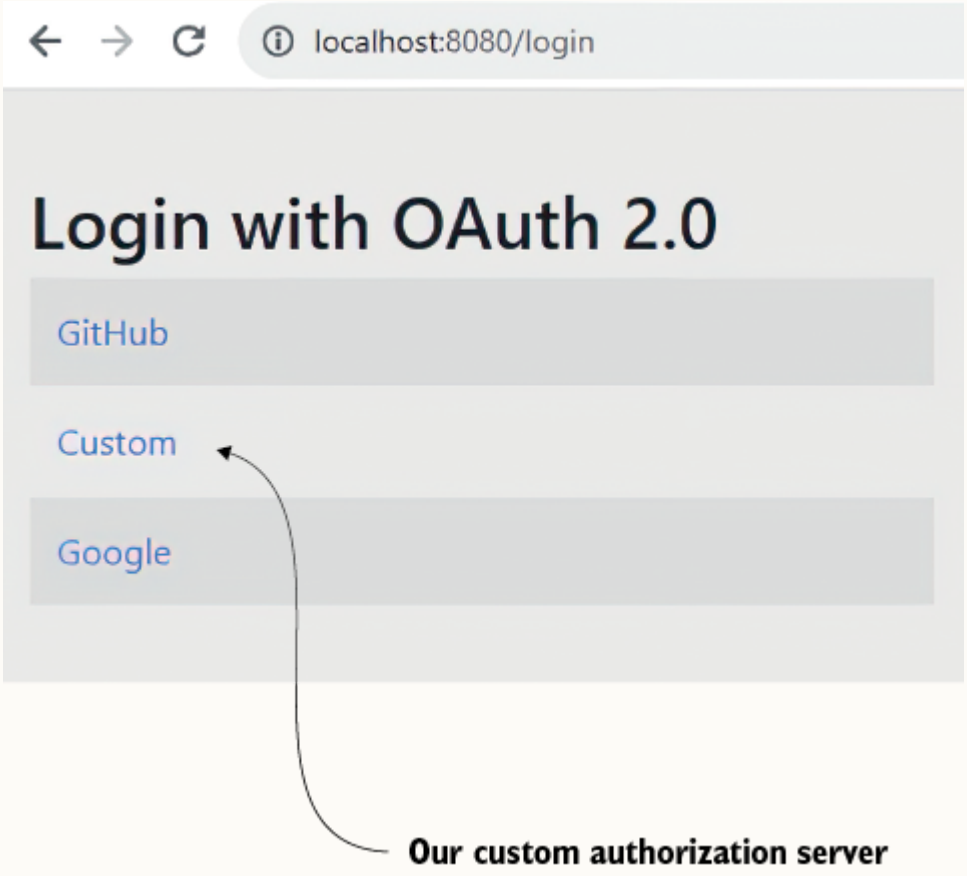



Figure 16.6 The custom authorization server now appears in the providers' list that the user can select from to authenticate.

### 16.1.4 Adding flexibility to your configurations

Often, we need to have more flexibility than the properties files offer us. Sometimes, we need to be able to dynamically change the credentials without redeploying the app. In other cases, we want to turn specific providers on or off or even offer access to these based on given logic. For such cases, adding the credentials in the properties file and allowing Spring Boot to do the magic for us doesn't work anymore.

However, if you know what happens behind the scenes, you can customize the provider's details as you wish. The only two types you must remember are

- `ClientRegistration` —This object is used to define the details the client needs to use the authorization server (credentials, redirect URI, authorization URI, etc.).
- `ClientRegistrationRepository` —This contract is implemented to define the logic that retrieves the client registrations. You, for example, can implement a client registration repository to tell your app to get the client registrations from a database or a custom vault.

For this example, I keep things simple. I'll continue using the application.properties file but with different names for the properties to prove that it's no longer Spring Boot configuring things for us. However, even if straightforward, this example shows the same

approach you'd use if you wanted to store the details in a database or grab them by calling a given endpoint. In any such case, you must appropriately implement the `ClientRegistrationRepository` contract.

You define the `ClientRegistrationRepository` component as a Spring bean. The app will use your implementation to get the client registration details. Listing 16.5 shows an example where I used an in-memory implementation. In this example, I do three things:

1. Inject the credentials values from the properties file
2. Create a `ClientRegistration` object with all the needed details
3. Configure it in an in-memory `ClientRegistrationRepository` implementation

You can find this example in project ssia-ch16-ex2.

**Listing 16.5 Implementing custom logic**

```
@Configuration
public class SecurityConfig {

 @Value("${client-id}")
 private String clientId;                                    ①

 @Value("${client-secret}")
 private String clientSecret;                                ①

 @Bean
 public SecurityFilterChain securityFilterChain(HttpSecurity http)
  throws Exception {

  http.oauth2Login(Customizer.withDefaults());

  http.authorizeHttpRequests(
   c -> c.anyRequest().authenticated()
  );

  return http.build();
 }

 @Bean
 public ClientRegistrationRepository clientRegistrationRepository() {
  return new InMemoryClientRegistrationRepository(
   this.googleClientRegistration());                         ②
 }
```

```
  private ClientRegistration googleClientRegistration() {
    return CommonOAuth2Provider.GOOGLE.getBuilder("google")      ③
            .clientId(clientId)
            .clientSecret(clientSecret)
            .build();
  }

}
```

① Injecting credentials values from the properties file

② Providing an in-memory repository implementation that contains the client registration details

③ Creating the client registration based on the template of the common provider Google

### 16.1.5 Managing authorization for an OAuth 2 login

In this section, we discuss using the authentication details. In most cases, your app needs to know who logged in. This requirement is either for displaying things differently or for applying various authorization restrictions. Fortunately, using the `oauth2Login()` authentication method doesn't differ in this regard from any other authentication method.

Remember the Spring Security authentication design we discussed starting with chapter 2 (reproduced as figure 16.7)? Successful authentication always ends with the app adding the authentication details to the security context. Using `oauth2Login()` is no exception.

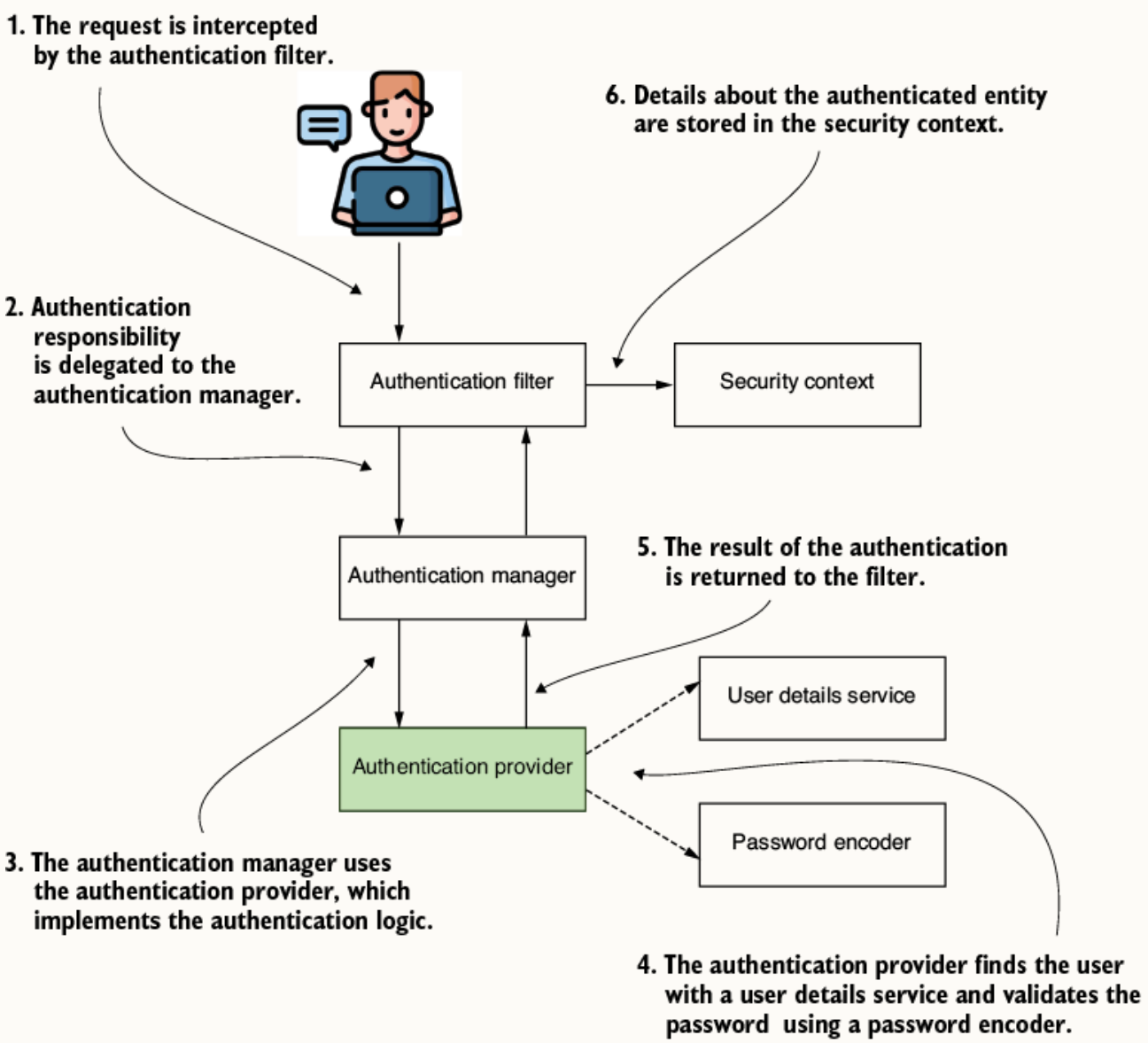



Figure 16.7 Authentication flow in Spring Security. Successful authentication ends with the app adding the details of the authenticated principal to the security context.

Knowing that the authentication details are in the security context, you can use them exactly the same way as for any other previously discussed authentication method—`httpBasic()`, `formLogin()`, or `oauth2ResourceServer()`:

- You can inject the `Authentication` object as a method parameter (figure 16.5).
- You can get it from the security context anywhere in the app (`SecurityContextHolder.getContext().getAuthentication()`).
- You can use pre-/post-annotations, as discussed in chapters 11 and 12.

You can use the `Authentication` contract to get standard user details such as the username and the authorities. If you need custom details, you can use the contract's implementation directly, as presented in listing 16.6. For OAuth 2, the class `OAuth2AuthenticationPrincipal` defines the contract implementation. Remember, however, that for maintainability purposes, I recommend you use the `Authentication` contract everywhere possible and rely on the implementation only if you have no other

choice (for example, if you need to get a detail you can't already get using the contract reference).

Listing 16.6 Obtaining the authentication details

```
@Controller
public class HomeController {

 @GetMapping("/")
 public String home(
  [CA]OAuth2AuthenticationToken authentication) {      ①
   // do something with the authentication
   return "index.html";
 }
}
```

① Injecting authentication details in the method's parameter

## 16.2 Implementing an OAuth 2 client

This section discusses implementing a service as an OAuth 2 client. In service-oriented systems, apps often communicate with one another. In such cases, the app sending the request to another app becomes a client of that particular app. In most cases, if we decide to implement authentication for the requests over OAuth 2, the app uses the client credentials grant type to obtain an access token.

The client credentials grant type doesn't imply a user. For this reason, you won't need a redirect URI and an authorization URI. The client credentials are enough to allow a client to authenticate and obtain an access token by sending a request to the token URI. Figure 16.8 reminds you of the client credentials grant type we discussed in chapter 13.

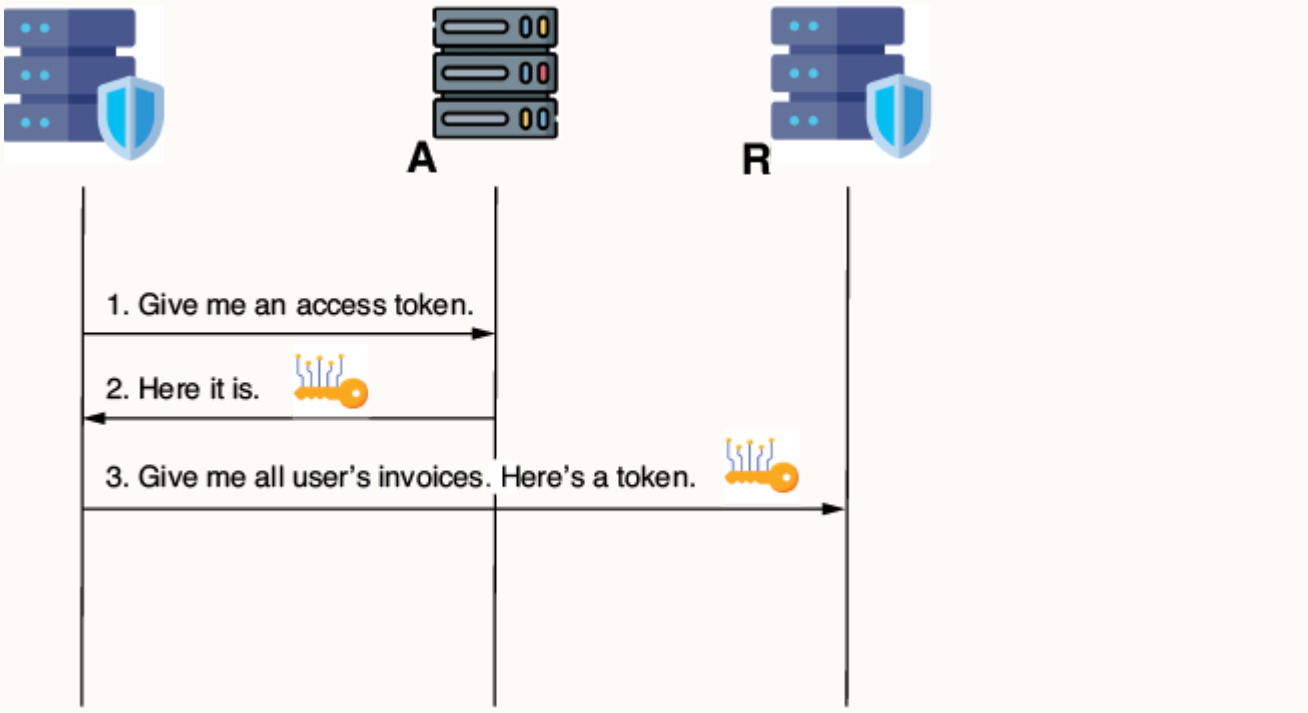



Figure 16.8 The client credentials grant type. The client sends a request to the token endpoint using the client credentials to authenticate. After successful authentication, the client receives an access token it can use to access resources on the resource server side.

Let's build a simple example to show you everything you need to know about implementing OAuth 2 client capabilities with Spring Security. We'll build an app that uses the client credentials grant type to get an access token from an authorization server. This app will get an access token from an authorization server. To simplify the example, we'll only discuss retrieving the access token. It's irrelevant for our demonstration of how you create the request. As long as you know how to get an access token, you can send the HTTP request either way, as any technology allows you to easily add a request header value (remember that you add the access token value to the `Authorization` request header prefixed with the string `"Bearer"`).

So what we'll precisely do in this example is configure an app to retrieve an access token from an OAuth 2 authorization server using the client credentials grant type. To prove that we correctly retrieved the access token, we'll return it in the response body of a demo endpoint. Figure 16.9 illustrates what we want to build. The steps shown in the figure are the following:

1. The user (you) calls a demo endpoint we named /token using cURL (or an alternative tool like Postman).
2. The tool (cURL) that simulates an app sends the request to the application we build for this example.
3. Our application uses the client credentials grant type to retrieve an access token from an authorization server.
4. The app returns the access token's value to the client in the HTTP response body.

5. The user (you) finds the access token value in the HTTP response body.

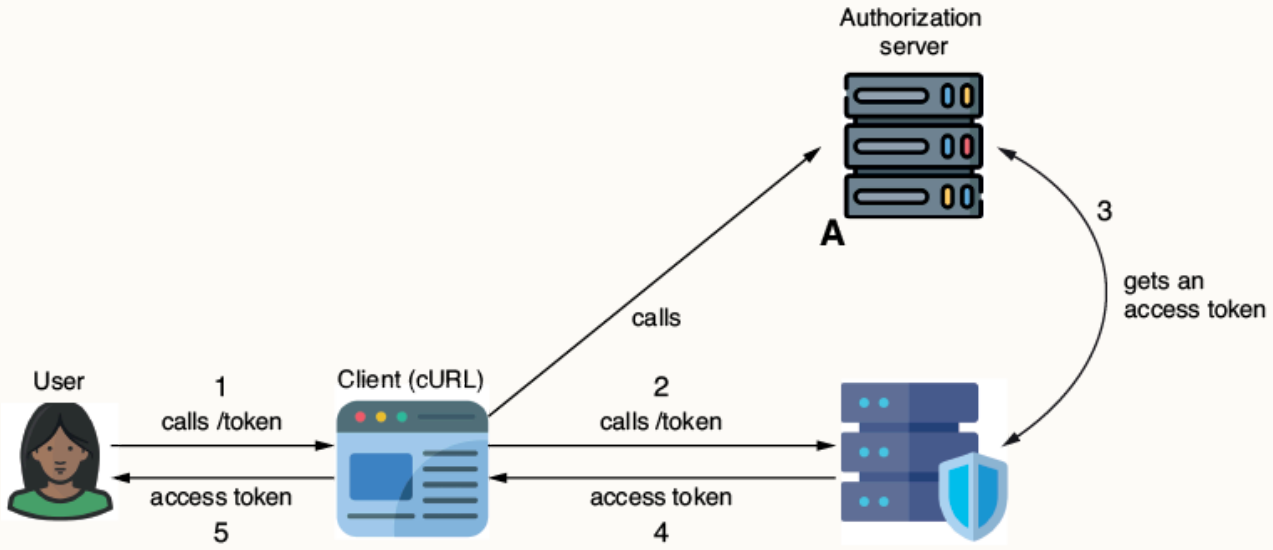



Figure 16.9 Our demonstration builds an app capable of retrieving an access token from an authorization server using the client credentials grant type. To prove that the app correctly retrieved the access token, the app sends the token value in response to a demo endpoint call. We named this demo endpoint /token.

We'll use the same authorization server you built in chapter 14, which you can find for this chapter in project ssia-ch16-ex1-as. Remember to first add to the authorization server a client registration that allows using the client credentials grant type. You can change the one you configured previously in chapter 14 (as presented in the next listing) or add a second client registration that fulfills this requirement.

**Listing 16.7 The client details registered on the authorization server side**

```
@Bean
public RegisteredClientRepository registeredClientRepository() {
  var registeredClient = RegisteredClient
    .withId(UUID.randomUUID().toString())
    .clientId("client")
    .clientSecret("secret")
    .clientAuthenticationMethod(
      ClientAuthenticationMethod.CLIENT_SECRET_BASIC)
    .authorizationGrantType(
      AuthorizationGrantType.CLIENT_CREDENTIALS)      ①
    .scope(OidcScopes.OPENID)
    .build();

    return new InMemoryRegisteredClientRepository(registeredClient);
  }
```

① Adding a client registration that allows the use of the client credentials grant type

Similarly to other authentication methods, Spring Security offers a method of the `HttpSecurity` object to configure an app as an OAuth 2 client. Call the `oauth2Client()` method presented in the following listing to configure the app as an OAuth 2 client.

**Listing 16.8 Configuring OAuth 2 client authentication**

```
@Configuration
public class ProjectConfig {

  @Bean
  public SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {

    http.oauth2Client(Customizer.withDefaults());      ①

    http.authorizeHttpRequests(
      c -> c.anyRequest().permitAll()
    );

    return http.build();
  }
}
```

① Using the oauth2Client() authentication method makes this app an OAuth 2 client.

The app also needs to know some details to send the authorization server the requests for access tokens. As you learned in section 16.1, we provide these details using a `ClientRegistrationRepository` component. You might find the code in listing 16.9 familiar, as it resembles the code we wrote in listing 16.4.

However, because I don't use a common provider, I had to specify more details, such as the scope, the token URI, and the authentication method. Observe that I configured client credentials as a grant type.

**Listing 16.9 Configuring the client registration details for the client app**

```
@Configuration
public class ProjectConfig {

  // Omitted code

  @Bean
```

```
  public ClientRegistrationRepository clientRegistrationRepository() {
    ClientRegistration c1 =
      ClientRegistration.withRegistrationId("1")
        .clientId("client")
        .clientSecret("secret")
        .authorizationGrantType(AuthorizationGrantType.CLIENT_CREDENTIALS)
        .clientAuthenticationMethod(
           ClientAuthenticationMethod.CLIENT_SECRET_BASIC)
        .tokenUri("http://localhost:7070/oauth2/token")
        .scope(OidcScopes.OPENID)
        .build();

    var repository =
       new InMemoryClientRegistrationRepository(c1);

    return repository;
  }
}
```

A client manager component makes the required request for obtaining an access token. Figure 16.10 illustrates the relationship between the controller and the client manager (for our example).

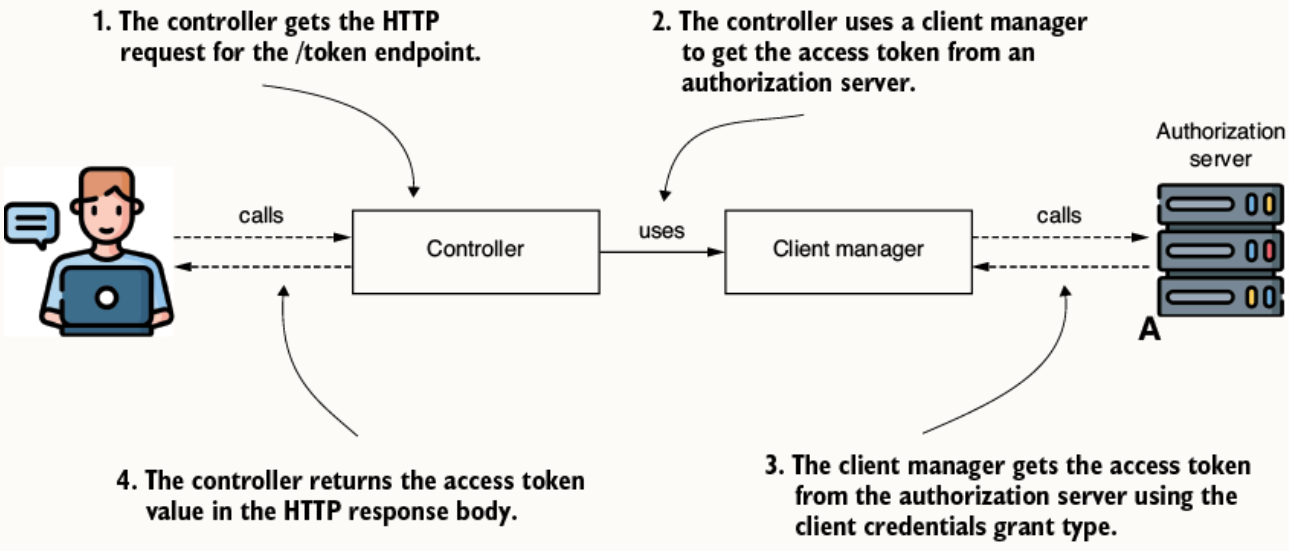



Figure 16.10 The controller uses a client manager to get the access token from an authorization server. The client manager is the Spring Security component responsible for connecting to the authorization server and properly using the grant type to get the access token.

The class `OAuth2AuthorizedClientManager` defines a client manager. The next listing configures a client manager as a bean in the app's context.

**Listing 16.10 Implementing an OAuth 2 client manager**

```
@Configuration
public class ProjectConfig {

 // Omitted code

 @Bean
 public OAuth2AuthorizedClientManager oAuth2AuthorizedClientManager(
  ClientRegistrationRepository clientRegistrationRepository,
  OAuth2AuthorizedClientRepository auth2AuthorizedClientRepository
 ) {

  var provider =                                    ①
     OAuth2AuthorizedClientProviderBuilder.builder()
        .clientCredentials()
        .build();

  var cm = new DefaultOAuth2AuthorizedClientManager(      ②
        clientRegistrationRepository,
        auth2AuthorizedClientRepository);

  cm.setAuthorizedClientProvider(provider);             ③

  return cm;
 }
}
```

① Creating a provider object to specify the grant types intended to be used

② Creating a client manager instance that will handle the client request logic

③ Setting the provider for the client manager

You can now use the client manager wherever you need to get an access token. As shown in figure 16.10, I made the controller use the client manager directly to simplify this example and allow you to focus on the discussion about implementing an OAuth 2 client. Remember that a real-world app would probably be more complex. In a design that correctly segregates the object's responsibilities, the client manager would likely be used by a proxy object and not directly by a controller (figure 16.11).

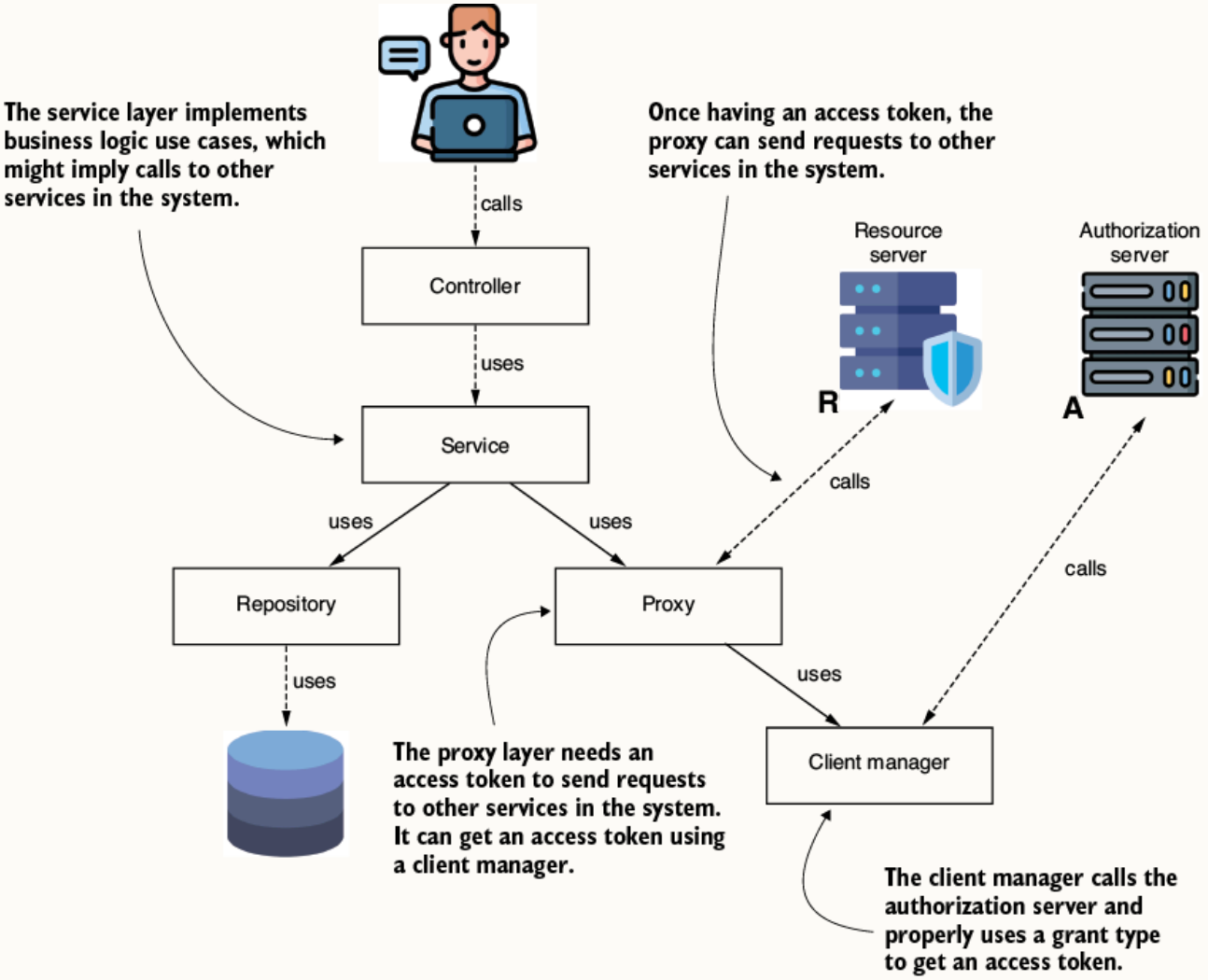



Figure 16.11 A real-world app would have better-separated responsibilities. Unlike our example, a proxy layer uses the token it obtains with the help of a client manager to send requests to another app in the system.

The next listing shows how to inject the client manager instance and demonstrates the retrieval of an access token using an endpoint. When calling the /token endpoint that the app exposes, the response body should contain the access token value.

**Listing 16.11 Using the OAuth 2 client manager to get a token**

```
@RestController
public class DemoController {

  private final OAuth2AuthorizedClientManager clientManager;

  // Omitted constructor

  @GetMapping("/token")                                    ①
  public String token() {
    OAuth2AuthorizeRequest request = OAuth2AuthorizeRequest
        .withClientRegistrationId("1")
        .principal("client")
        .build();                                          ②
```

```
    var client =
      clientManager.authorize(request);          ③

    return client
      .getAccessToken().getTokenValue();         ④
  }
}
```

① Exposing a GET endpoint at the /token path

② Creating an authorization request instance

③ Sending the request, the app returns the access token value.

④ The app returns the access token value in the response body.

Use the following cURL command to call the endpoint that the app exposes:

```
curl http://localhost:8080/token
```

The response body should contain the value of an access token, similar to

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6Im
JpbGwiLCJpYXQiOjE1MTYyMzkwMjJ9.zjL2JXwOTVgNgTMUKmPO-PTPklULUVmV_5re5OeZoHw
```

## Summary

- When implementing a Spring web app, we must often configure authentication capabilities. While we can implement a login form quickly with the `formLogin()` method, we can also allow users to authenticate using another system with a registered account.
- Allowing users to choose a different system to log in offers advantages for both the user and our app. The users don't need to remember supplementary credentials, and our app doesn't have to manage credentials for all their users.
- Spring Security considers GitHub, Google, Facebook, and Okta common providers. For the common providers, Spring Security already knows all the details to establish requests over the OAuth 2 framework, so you only need to configure the client credentials the provider offers to configure the login capability.

- You can configure your app to use providers other than the common ones, but you need to explicitly configure all the details the app needs to establish the grant type flows to get access tokens. The main details you need to configure are three URIs: the authorization, the token, and the key set URI.
- Once the user logs into your app, even if it is authenticated through an external system, the app gets details about them, storing the details in the security context. This process follows the standard Spring Security authentication design. For this reason, you can configure authorization similarly to all the other authentication methods.
- Sometimes, a backend service becomes a client for another backend app. In such a case, an app that wants to call another app and use an OAuth 2 approach needs to get an access token to be authenticated by the latter. A service can use the client credentials grant type to get an access token.
- Spring Security provides an object called a client manager. This object implements the logic for executing a specific grant type and getting an access token. An app's proxy layer that sends requests to another app and needs to authenticate the requests using access tokens would use a client manager to get the access token.

Previous chapter
< 15 Implementing an OAuth 2 resource server

Next chapter
Part 5. Going reactive >